SYSTEM MAP

Drumbeat — Architecture

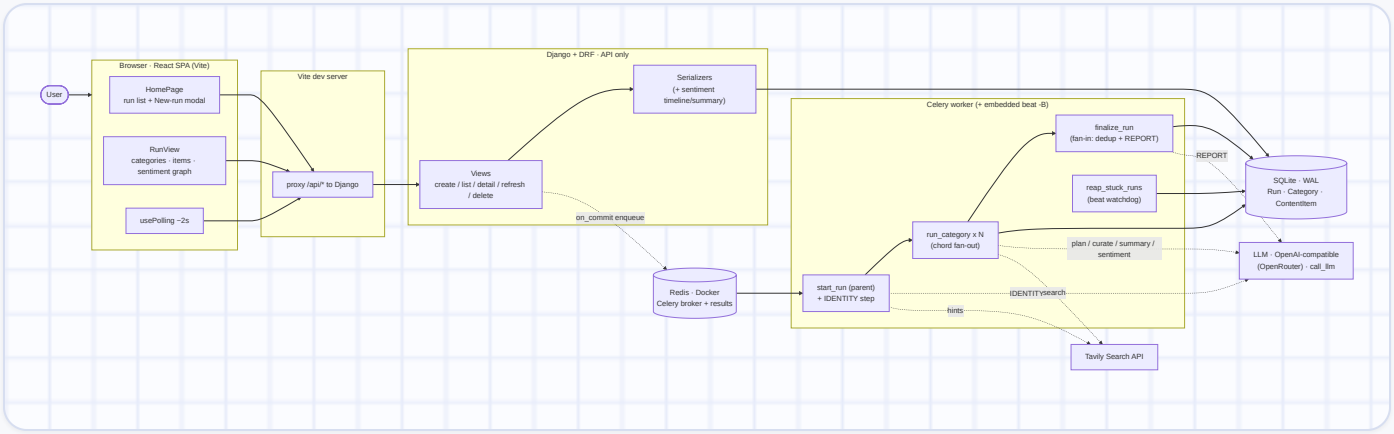
A locally-run app that, given a company name or URL, runs a background job to find third-party content about it, scores each item's sentiment, and presents a categorised, reviewable list with a sentiment trend.

`Django + DRF` `Celery · Redis` `SQLite (WAL)` `React + Vite` `OpenRouter LLM` `Tavily`

---

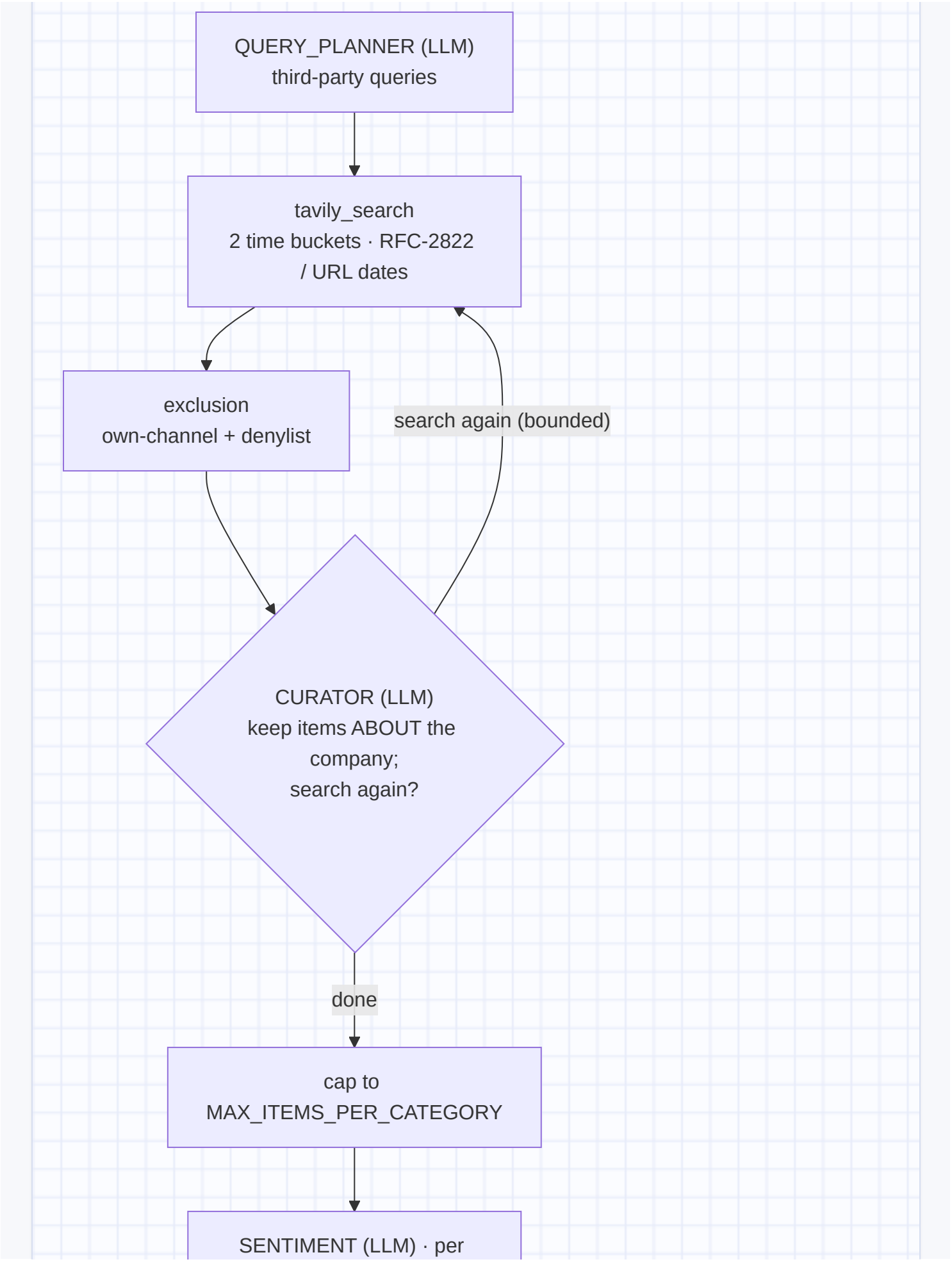
§1 System / process view

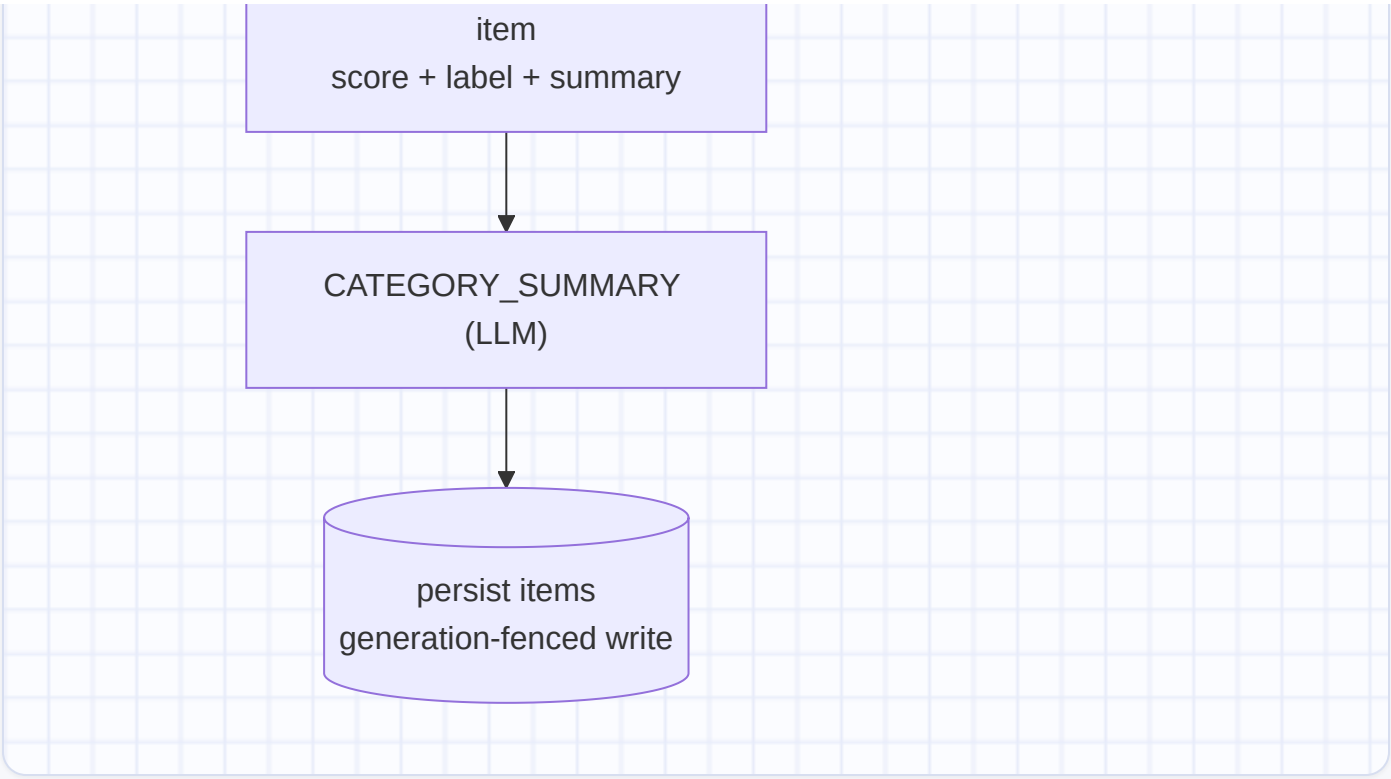
Four processes start via `./start_all.sh`, each on a runtime-discovered port. The React SPA talks only to the DRF JSON API; all research runs in a Celery worker; state lives in SQLite; Redis is the Celery broker and result backend.



§2 Per-category research pipeline

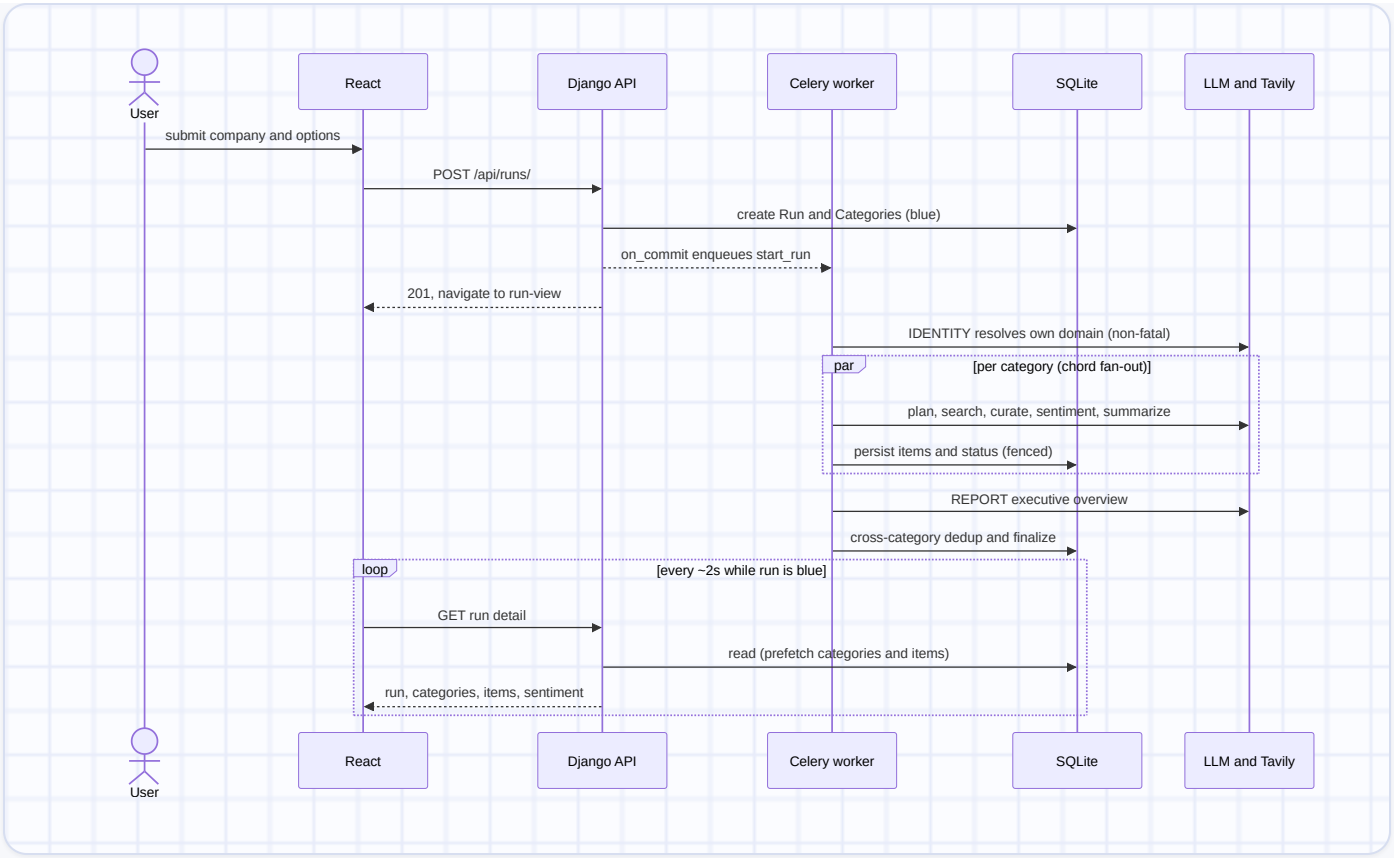
Each `run_category` subtask runs `research_category` (pure, no DB), then persists its result in one generation-fenced write. The curator may request more searches, bounded by `CURATOR_MAX_ITERATIONS` / `CURATOR_MAX_SEARCHES`. Every LLM call goes through one `call_llm` with retry and a non-fatal fallback.


QUERY_PLANNER (LLM)
third-party queries
tavily_search
2 time buckets · RFC-2822
/ URL dates
exclusion
own-channel + denylist
search again (bounded)
CURATOR (LLM)
keep items ABOUT the
company;
search again?
done
cap to
MAX_ITEMS_PER_CATEGORY
SENTIMENT (LLM) · per



§3 Run lifecycle

A run is a Celery **chord**: an IDENTITY step, one subtask per selected category (fan-out), then a fan-in callback. Correctness under refresh / delete / timeout rests on generation-fenced writes, not task revocation. The frontend learns progress by polling.



§4 Key invariants

Why the system is shaped the way it is.

● green · complete ● yellow · partial ● red · failed ● blue · running

> **Generation fencing**
>
> Every task DB write is guarded by a compare-and-set on `Run.generation`; a superseded write raises and rolls back. Refresh and the reaper bump the generation — this, not `revoke`, prevents stale-task clobber.

> **Never hold a transaction across an LLM/network call**
>
> The fan-in computes the dedup plan and calls REPORT outside any transaction, then does one short fenced write — so the single SQLite writer lock never blocks polls or sibling categories.

> **Fail loud, with scoped non-fatal exceptions**
>
> Missing config stops startup; bad API input returns a 400. IDENTITY, per-item SENTIMENT, and each LLM call-point degrade non-fatally (retry, then fallback), so one bad item or reply never fails a whole category or run.

Status measures completeness, not abundance

Green = all categories finished cleanly and ≥ 1 item exists; yellow = some category errored or timed out; red = zero items or all errored; blue = in progress.

Live updates by honest polling

A category leaves "running" only after its items, summary, and `ended_at` commit; a run leaves "blue" only after the overview and `ended_at` commit — so a ~2s poll never sees a half-written run.

§5 Where things live

CONCERN	MODULE
API views / serializers	`research/views.py · serializers.py`
Models (SQLite)	`research/models.py`
Celery tasks / orchestration	`research/tasks.py · drumbeat/celery.py`
Per-category pipeline	`research/pipeline.py`
LLM seam + retry / schemas	`research/llm.py · schemas.py`
Search seam	`research/tavily.py`
URL identity / exclusion	`research/urls_util.py · exclusion.py`
Generation fencing	`research/fencing.py`
Frontend	`frontend/src/components/* · usePolling.js`
Local orchestration	`start_all.sh · run_tests.sh`

Source of truth: `plans/INITIAL.md`, `plans/SENTIMENT-DESIGN.md`, and `docs/ARCHITECTURE.md`. Diagrams render on a fixed light canvas for legibility in either theme.